```
        driver.findElement(By.name("email")).sendKeys(userEmail);
        driver.findElement(By.name("password")).sendKeys(userPassword);
        driver.findElement(By.id("signin")).click();
    }
}
```

❺ Web element selector details.

This class wraps the lines of code you saw earlier ❺ into a single method called `signinWithCredentials()` ❹. It also provides an `open()` ❸ method to get to the right page. The code needs a `WebDriver` instance ❶, which is provided in the constructor ❷.

Now your test code can focus on the test data and the intent of the actions, rather than on how the individual web elements are found or manipulated:

```
LoginPage loginPage = new LoginPage(driver);
loginPage.open();
loginPage.signInWithCredentials("jane.smith@acme.com", "s3cr3t");
```

Open the login page.

Sign in with the specified credentials.

8.3.1 Introducing the Page Objects pattern

The class we just looked at follows the Page Objects pattern. A page object is a class that models a web page, or part of a web page, and that presents a set of business-focused methods for tests to use, sparing them from the implementation details of the actual HTML page. A page object has two main roles:

- It isolates the technical implementation of the page from the tests, making the test code simpler and easier to maintain.
- It centralizes the code that interacts with a page (or a page component), so that when the web page is modified, the test code only needs to be updated in one place.

In section 7.3, we discussed the typical layers that make up a well-structured automated acceptance test suite:

- The *Business Rules* layer describes the expected business outcomes.
- The *Business Flow* layer relates the user's journey through the application.
- The *Technical* layer interacts directly with the system.

Page objects belong in the Technical layer (see figure 8.12). They provide business-friendly services to the Business Flow layer and implement the interactions with the web pages using the WebDriver API.

Page objects don't have to represent an actual page. In some cases, such as the login page discussed earlier, it makes sense to have a page object dedicated to a page. In other cases, such as a modern JavaScript single-page application, a single HTML page might be represented by page objects for each state or view of the application.

It also makes sense to use page objects to represent important parts of a screen, particularly if those parts are reused from screen to screen. For example, you might use a page object to represent the main menu bar that appears on every screen, or for the list of featured destinations if this appears in several places. These component objects can be nested within other page objects or used independently.